SE Lab 3 Contents

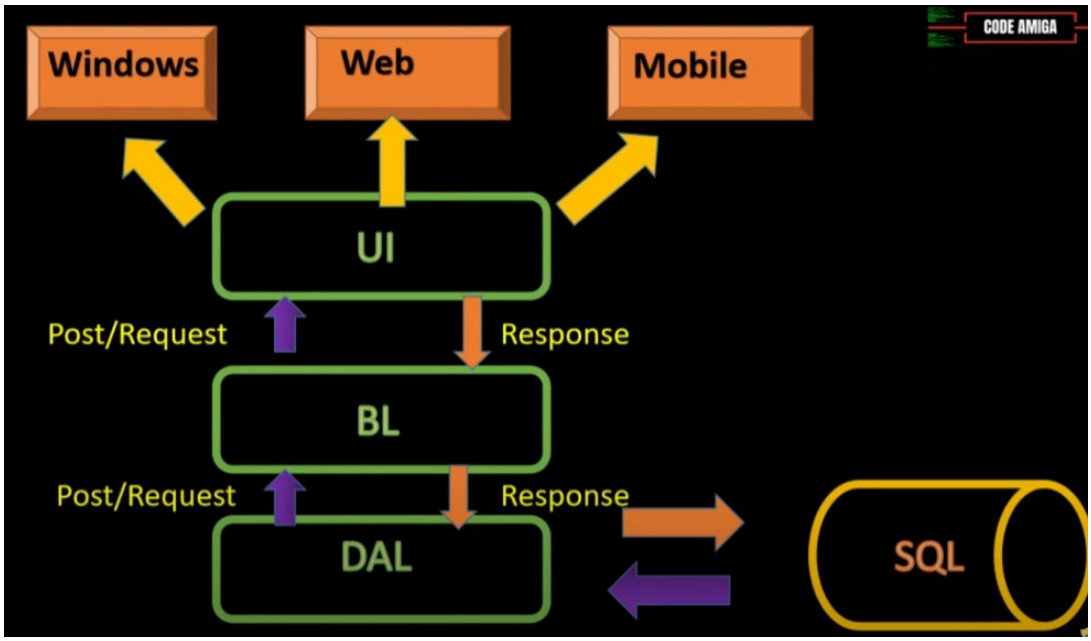


Image Source: https://www.youtube.com/watch?v=P4E9vHYsyAU

I. Creating a 3-Tier Architecture in C# .NET

Involves separating your application into three distinct layers: the Presentation Layer, the Business Logic Layer, and the Data Access Layer. Here's a step-by-step guide to help you get started:

1. Presentation Layer

This layer is responsible for the user interface and user interaction. **In a .NET application, this could be an ASP.NET MVC project or a Windows Forms/WPF application.**

- **Create a new project**: Start by creating a new ASP.NET MVC or Windows Forms/WPF project.
- **Design the UI**: Add views, controllers, or forms to handle user input and display data.

2. Business Logic Layer (BLL)

This layer contains the core functionality and business rules of your application.

- **Create a Class Library**: Add a new Class Library project to your solution for the Business Logic Layer.
- **Implement Business Logic**: Create classes and methods to handle the business rules and operations. For example, you might have a CustomerService class with methods like AddCustomer, GetCustomer, etc.

3. Data Access Layer (DAL)

This layer is responsible for interacting with the database.

- **Create another Class Library**: Add a new Class Library project for the Data Access Layer.

- **Implement Data Access**: Use Entity Framework or ADO.NET to interact with the database. Create repository classes to handle CRUD operations. For example, you might have a CustomerRepository class with methods like AddCustomer, GetCustomerById, etc.

4. Introduction to Connecting Windows Forms (C#) to MSSQL Using Entity Framework

Connecting a Windows Forms application to a Microsoft SQL Server (MSSQL) database is a common requirement for many desktop applications. Entity Framework (EF) is an Object-Relational Mapper (ORM) that enables .NET developers to work with a database using .NET objects, eliminating the need for most of the data-access code that developers usually need to write. In this introduction, we will cover the basics of connecting a Windows Forms application to an MSSQL database using Entity Framework. We will use the Entity Framework wizard to generate the necessary classes and context, and we will also discuss how to configure the connection string.

4. Steps to Connect Windows Forms to MSSQL Using Entity Framework

4.1. Set Up Your MSSQL Database:

- Ensure you have an MSSQL database set up and accessible. You can use SQL Server Management Studio (SSMS) to create and manage your database.
- Create a sample database and table for this exercise.

4.2. Create a New Windows Forms Project:

- Open Visual Studio and create a new Windows Forms App (.NET Framework) project.
- Name your project and choose a location to save it.

4.3. Install Entity Framework (used from Lab 3)

- Right-click on your project in the Solution Explorer and select Manage NuGet Packages.
- Search for EntityFramework and install it.

4.4. Use the Entity Framework Wizard (used from Lab 3):

- Right-click on your project in the Solution Explorer, select Add > New Item.
- Choose Data from the left menu and select ADO.NET Entity Data Model. Name it and click Add.
- In the Entity Data Model Wizard, select EF Designer from database and click Next.

- Choose your data connection or create a new connection to your MSSQL database. This will automatically generate the connection string in your App.config file.
- Select the database objects (tables, views, stored procedures) you want to include in your model and click Finish.

4.5. Configure the Connection String:

- The connection string is automatically added to your App.config file when you create the Entity Data Model. It looks something like this:
- <connectionStrings>
- <add name="YourEntities" connectionString="metadata=res://*/YourModel.csdl|res://*/YourModel.ssdl|res://*/YourModel.msl;provider=System.Data.SqlClient;provider connection string="data source=YourServer;initial catalog=YourDatabase;integrated security=True;MultipleActiveResultSets=True;App=EntityFramework"" providerName="System.Data.EntityClient" />
- </connectionStrings>
- Ensure the connection string points to your MSSQL server and database.

4.6. Using the Generated Context and Classes:

- The Entity Framework wizard generates a context class and entity classes based on your database schema.
- You can use these classes to interact with your database. For example, to retrieve data from a table:

```
using (var context = new YourEntities())
{
    var data = context.YourTable.ToList();
    // Bind data to a control, e.g., DataGridView
    dataGridView1.DataSource = data;
}
```

4.7. Summary

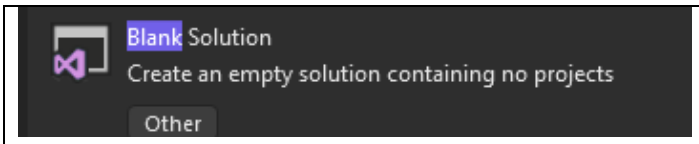
By following these steps, you can easily connect your Windows Forms application to an MSSQL database using Entity Framework. The Entity Framework wizard simplifies the process by generating the necessary classes and context, while the connection string in the App.config file ensures your application can communicate with the database. This approach allows you to focus on building your application's functionality without worrying about the complexities of database access code.

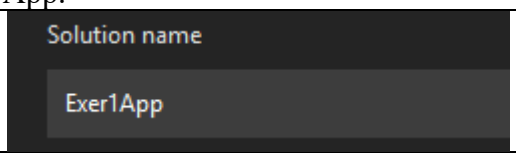
II. Class Activity

Exercise 1: Setting up 3-Tier Architecture with WinformUI

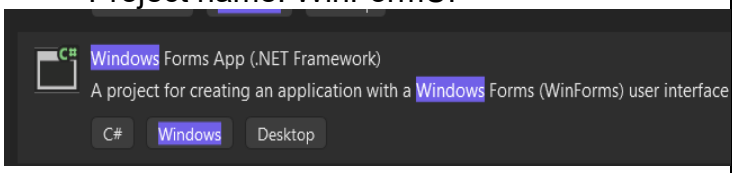
Step 1: Set Up the Solution

1. **Open Visual Studio** and create a new solution:
    - Go to File > New > Project.
    - Select Blank Solution and name it Exer1App.

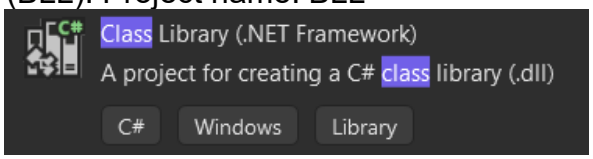




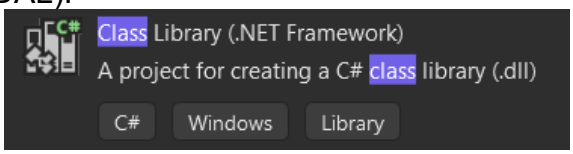
2. **Add three projects** to the solution:
    - **Windows Forms Application(.Net Framework)** for the Presentation Layer. Project name: WinFormUI



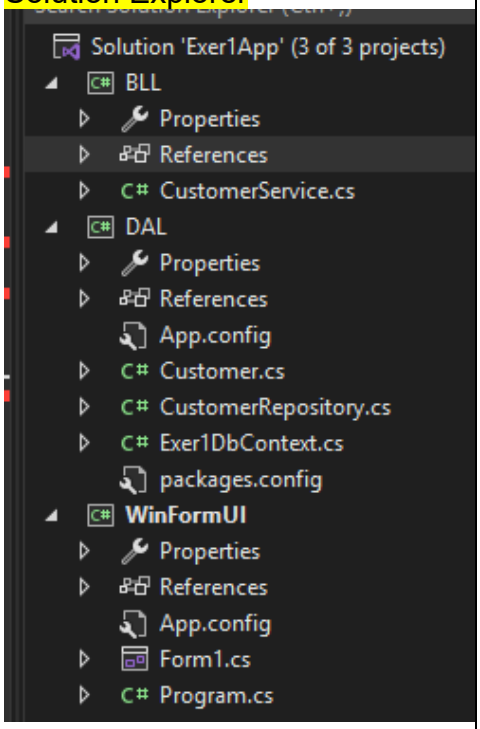
    - **Class Library** for the Business Logic Layer (BLL). Project name: BLL



    - **Class Library** for the Data Access Layer (DAL).



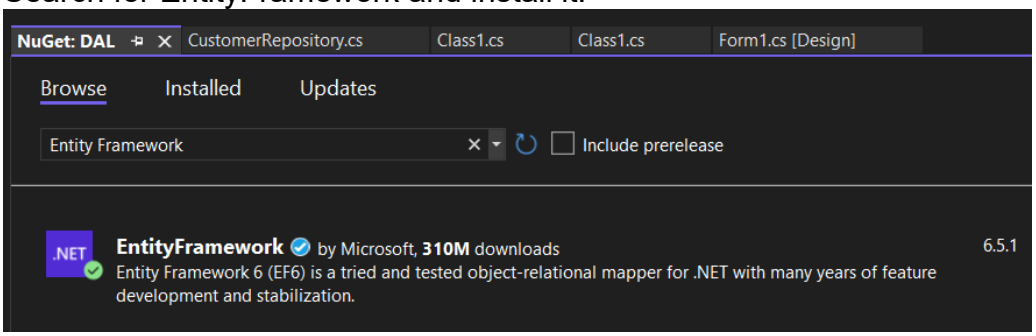
Solution Explorer



Step 2: Create the Data Access Layer (DAL)

1. **Add a new Class Library project** named DAL:
    - Right-click on the solution in Solution Explorer.
    - Select Add > New Project.
    - Choose Class Library (.NET Framework) and name it DAL.

2. **Add a class** named CustomerRepository to handle database operations:
    - Right-click on the DAL project.
    - Select Add > Class and name it CustomerRepository.cs.
3. **Install Entity Framework** via NuGet Package Manager:
    - Right-click on the DAL project.
    - Select Manage NuGet Packages.
    - Search for EntityFramework and install it.



Example Code for CustomerRepository

// Customer.cs (Model)

```
public class Customer
{
    public int Id { get; set; } //CustomerId
    public string Name { get; set; }
    public string Email { get; set; }
}
```

// Exer1DbContext.cs

```
public class Exer1DbContext : DbContext
{
    public DbSet<Customer> Customers { get; set; }
    public Exer1DbContext() : base("name=MyConn")
    { }
}
```

// CustomerRepository.cs

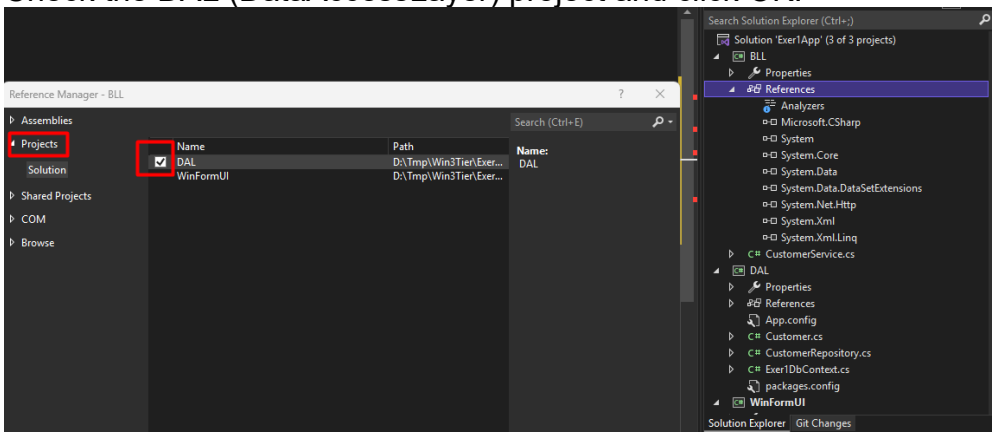
public class CustomerRepository

```
{
    private readonly Exer1DbContext _context;
    public CustomerRepository()
    {
        _context = new Exer1DbContext();
    }
    public void AddCustomer(Customer customer)
    {
        _context.Customers.Add(customer);
        _context.SaveChanges();
    }
    public Customer GetCustomerById(int id)
    {
        return _context.Customers.FirstOrDefault(c => c.CustomerID == id);
    }
    public List<Customer> GetAllCustomers()
    {
        return _context.Customers.ToList();
    }
    public void UpdateCustomer(Customer customer)
    {
        var existingCustomer = _context.Customers.FirstOrDefault(c => c.CustomerID ==
customer.CustomerID);
        if (existingCustomer != null)
        {
            existingCustomer.Name = customer.Name;
            existingCustomer.Email = customer.Email;
            _context.SaveChanges();
        }
    }
    public void DeleteCustomer(int id)
    {
        var customer = _context.Customers.FirstOrDefault(c => c.Id == id);
        if (customer != null)
        {
```

```
            _context.Customers.Remove(customer);
            _context.SaveChanges();
        }
    }
}
```

Step 3: Create the Business Logic Layer (BLL)

1. **Add a new Class Library project** named BLL (BusinessLogicLayer):
    - Right-click on the solution in Solution Explorer.
    - Select Add > New Project.
    - Choose Class Library (.NET Framework) and name it BusinessLogicLayer.
2. **Add a class** named CustomerService to handle business logic:
    - Right-click on the BusinessLogicLayer project.
    - Select Add > Class and name it CustomerService.cs.
3. **Reference the Data Access Layer** project in the BLL(Business Logic Layer) project:
    - Right-click on the BusinessLogicLayer project.
    - Select Add > Reference.
    - Check the DAL (DataAccessLayer) project and click OK.



Right click on DAL and select Build

Example Code for CustomerService

```
// CustomerService.cs
    public class CustomerService
    {
```

```
    private readonly CustomerRepository _customerRepository;

    public CustomerService()
    {
        _customerRepository = new CustomerRepository();
    }

    public void AddCustomer(Customer customer)
    {
        // Business logic before adding customer
        _customerRepository.AddCustomer(customer);
    }

    public Customer GetCustomer(int id)
    {
        // Business logic before retrieving customer
        return _customerRepository.GetCustomerById(id);
    }
public List<Customer> GetAllCustomers()
    {
        return _customerRepository.GetAllCustomers();
    }

    public void UpdateCustomer(Customer customer)
    {
        // Business logic before updating customer
        _customerRepository.UpdateCustomer(customer);
    }

    public void DeleteCustomer(int id)
    {
        // Business logic before deleting customer
        _customerRepository.DeleteCustomer(id);
```

```
    }
}
```

Step 4: Create the WinformUI (Presentation Layer)

1. **Add a new Windows Forms Application project** named PresentationLayer:
    - Right-click on the solution in Solution Explorer.
    - Select Add > New Project.
    - Choose Windows Forms App (.NET Framework) and name it WindowFormUI (or PresentationLayer) .
2. **Reference the Business Logic Layer** project in the Presentation Layer project:
    - Right-click on the **WinformUI** (PresentationLayer) project.
    - Select Add > Reference.
    - Check the **BLL** (BusinessLogicLayer) project and click OK.
    - **Install Entity Framework** via NuGet Package Manager:
        - Right-click on the DAL project.
        - Select Manage NuGet Packages.
        - Search for EntityFramework and install it.
3. **Design the UI**: Add forms and controls to handle user input and display data:
    - Open Form1.cs and design the form with text boxes, labels, and buttons for adding and retrieving customers.
      (Add a new form or rename **Form1.cs** to **CustomerForm.cs**)
    - **Design the Form**
        1. **Open CustomerForm.cs** in the designer view.
        2. **Add controls** to the form:
            - **Labels**: For "Name", "Email", and "Customer ID".
            - **TextBoxes**: For entering "Name", "Email", and "Customer ID".
            - **Buttons**: For "Add Customer" and "Get Customer".
    - **Set Up the Controls**
        1. **Drag and drop** the controls from the Toolbox onto the form.
        2. **Set the properties** of the controls:
            - **Labels**: Set the Text property to "Name", "Email", and "Customer ID".

- **TextBoxes**: Name them txtName, txtEmail, and txtCustomerId.
- **Buttons**: Name them btnAddCustomer and btnGetCustomer, and set the Text property to "Add Customer" and "Get Customer".
- May add datagridview to show all data from Customer table

- **Add Event Handlers**

1. **Double-click** on the "Add Customer" button to create an event handler for the Click event.
2. **Double-click** on the "Get Customer" button to create an event handler for the Click event.
3. Double-click on the Customer Form to create CustomerForm_Load event

- **Implement the Code**

1. **Open `CustomerForm.cs`** in code view.
2. **Add the necessary using directives**:

```
using BLL; //using BusinessLogicLayer;
using DAL; //using DataAccessLayer;
```

3. **Add a private field** for the `CustomerService`:

```
private readonly CustomerService _customerService;

public CustomerForm()
{
    InitializeComponent();
    _customerService = new CustomerService();
}
```

4. **Implement the `btnAddCustomer_Click` event handler**:

```
private void btnAddCustomer_Click(object sender,
EventArgs e)
{
    var customer = new Customer
    {
        Name = txtName.Text,
        Email = txtEmail.Text
    };
    _customerService.AddCustomer(customer);
    MessageBox.Show("Customer added successfully!");
}
```

5. **Implement the `btnGetCustomer_Click` event handler**:

```
private void btnGetCustomer_Click(object sender,
EventArgs e)
{
    int customerId = int.Parse(txtCustomerId.Text);
    var customer =
_customerService.GetCustomer(customerId);
    if (customer != null)
    {
        txtName.Text = customer.Name;
        txtEmail.Text = customer.Email;
    }
    else
    {
        MessageBox.Show("Customer not found!");
    }
}
```

Code to get data to datagridview

```
private void CustomerForm_Load(object sender, EventArgs e)
{
    LoadCustomers();
}
private void LoadCustomers()
{
    var customers = _customerService.GetAllCustomers();
    dataGridView1.DataSource=customers;
}
```

6. **Set `CustomerForm` as the startup form**:

   - Open `Program.cs` in the `PresentationLayer` project.
   - Modify the `Main` method to start `CustomerForm`:

```
Application.Run(new CustomerForm());
```

Step 5: Configuration for Database Connection to MSSQL

Make sure to configure the connection string in your App.config (under WinformUI project) or Web.config file for the Exer1DbContext.cs to connect to your database.

```
<connectionStrings>
    <add name="MyConn"
        connectionString="Data Source=(localdb)\MSSQL2019;Initial Catalog=LabDB;Integrated Security=True"
        providerName="System.Data.SqlClient" />
</connectionStrings>
```

You should store the Exer1DbContext.cs file in the DAL (Data Access Layer) project. Here's how you can do it:

1. **Add the Exer1DbContext.cs file** to the **DAL** (DataAccessLayer) project:
   - Right-click on the **DAL**(DataAccessLayer) project in Solution Explorer.
   - Select Add > Class (*or rename Class1.cs*)
   - Name the class **Exer1DbContext.cs.**
2. **Add the following code** to the Exer1DbContext.cs file:

```
using System.Data.Entity;
public class Exer1DbContext: DbContext
{
   public DbSet<Customer> Customers { get; set; }
   public Exer1DbContext () : base("name=MyConn")
   {
   }
}
```

This will ensure that your **Exer1DbContext** class is part of the **DAL**(Data Access Layer), where it can manage the database connection and entity sets.

Step 6: Create table Customer with MSSQL

- Create database name : LabDB, create table Customer

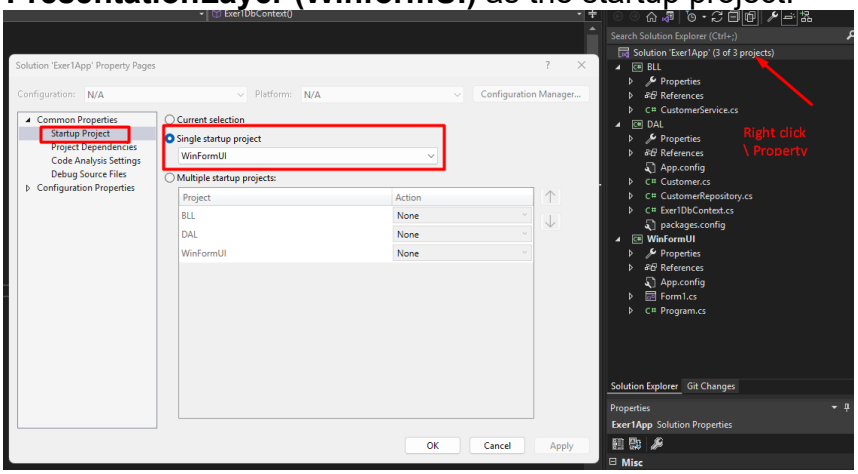
```
CREATE TABLE Customers (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100),
    Email NVARCHAR(100)
);
```

Sample data into the Customers table:

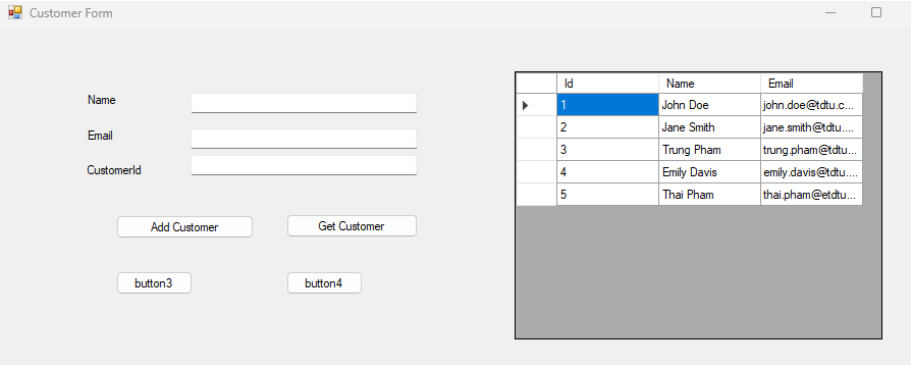
```
INSERT INTO Customers (Name, Email) VALUES ('John Doe', 'john.doe@tdtu.com');
INSERT INTO Customers (Name, Email) VALUES ('Jane Smith', 'jane.smith@tdtu.com');
INSERT INTO Customers (Name, Email) VALUES ('Trung Pham', 'trung.pham@tdtu.com');
INSERT INTO Customers (Name, Email) VALUES ('Emily Davis', 'emily.davis@tdtu.com');
INSERT INTO Customers (Name, Email) VALUES ('Thai Pham', 'thai.pham@etdtu.com');
```

Step 7: Run the Application

1. **Build the solution** to ensure all projects compile successfully:
    - Go to Build > Build Solution.
2. **Run the Windows Forms application** and test the functionality:
    - Set **PresentationLayer (WinformUI)** as the startup project.



    - Press F5 to run the application.



This should give you a basic Windows Forms application using a 3-Tier Architecture. You can expand on this by adding more features, implementing dependency injection, and improving error handling.

Exercise 2: Login Form with 3-Tier Architecture

- Change Login form in Lab2 to 3-Tier Architecture
- Should create a new project with setting up from scratch like Exer1

Exercise 3: Login Form to Main form with 3-Tier Architecture

- Use Exer2, after logging in successfully then go to Main Form.
- On Main form, add menu strip with link to Customer form (reuse Exer 1)
- Also add Product Form, Supplier Form

III. Homework

-Use the same requirements from **Lab2 (School Management System)** homework but you should apply 3-Tier Architecture.

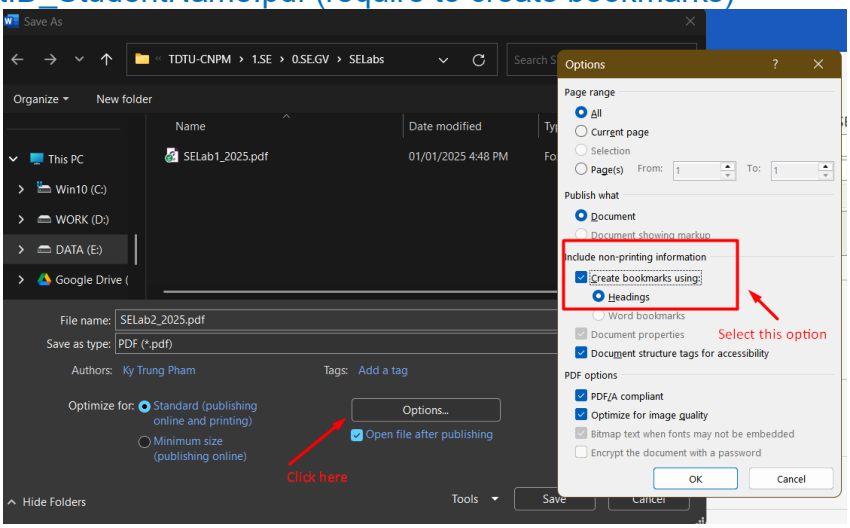
IV. Lab 3 Report Submission

- **Introduction:** Briefly describe the purpose of the exercises.
- **Exercises:** List each exercise with a brief description of what you did and learned for Class Activity included the screenshots of running App output.
- **Homework:** Describe what you do with including the screenshots of the output for homework exercise.
- **Conclusion:** Summarize your overall learning experience and any challenges you faced.

By following these exercises, you will learn how to create a 3-Tier Architecture Application, connect to an MSSQL database

Submit 2 files on eLearning:

- StudentID_StudentName.zip (contain Windows Form Project & .sql file) for both Class Activity and Homework.
- StudentID_StudentName.pdf (require to create bookmarks)



Notes: There may be a typo/mistake during creating this document. Please correct it by yourself.

Enjoy ☺ Email: tg_phamthaikytrung@tdtu.edu.vn